Movement AI Process Letter

The process of completing this assignment involved lecture notes, slides, and the Game AI textbook. Using the steering behaviors from class, I built my solutions from the pseudo-code and then adjusted them through testing. I used a Rigidbody struct to handle integration, orientation matching, speed limits, and screen wrapping. Having this centralized made it easier to change behavior without rewriting every algorithm. As I worked, I realized how important tunable values were. Small tweaks to radii, weights, or rates could completely change how things looked, so development became more about experimentation than just implementation.

One of the biggest improvements to my workflow was adding an enum class that allowed me to switch modes with keys. Instead of commenting out code, I could instantly compare kinematic motion, seek, wander, and flocking. This made debugging and tuning much easier and helped me understand the differences between behaviors.

```
// ---------- Movement Modes ----------
enum class MoveMode {
    KINEMATIC,
    SEEK,
    WANDER,
    FLOCKING
};
```

```
case MoveMode::KINEMATIC:   active = "EDGE"; break;
case MoveMode::SEEK:        active = "SEEK"; break;
case MoveMode::WANDER:      active = "WANDER"; break;
case MoveMode::FLOCKING:    active = "FLOCKING"; break;
}
```

The first task was drawing the boid, moving it along the screen edges, and leaving breadcrumbs. For this kinematic behavior, I directly assigned velocity and changed direction whenever the boid hit a screen margin. Because velocity was set explicitly, turns were sharp and immediate. Breadcrumbs helped confirm the path and made it obvious when wrapping or movement logic worked incorrectly.

For Dynamic Seek, it was a little different, since this time I needed to calculate the position and distance of the boid from a target, which was wherever the player clicked their mouse. Instead of directly setting velocity like in the kinematic example, I computed a desired velocity using position matching toward the target and then used velocity matching to gradually adjust the boid's movement. Orientation was derived from the velocity of each frame, so the boid naturally faced the direction it was traveling.

Part of my polish phase was adding a visible red target marker to give feedback to the player and make it obvious what the boid was reacting to. For arrival, I experimented with two approaches. The proportional slow radius method created a smoother, more natural stop, while the

time-to-target style reacted faster but could feel more sudden. Seeing both made me realize that different methods can be correct but produce very different feelings in motion.

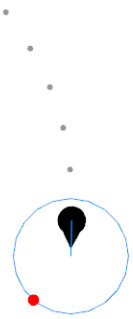
The image on the left showcases the boid calculating its path toward the target, with the right image showing that it has slowed and come to rest after entering the slow radius.



Next, I worked on Wander. I followed the pseudo-code from the book and added the projected circle and target as debug visuals. Seeing this made the behavior much easier to understand. Wander is really just a seek applied to a point that shifts slightly over time. Because the randomness affects the target instead of the velocity directly, the boid moves smoothly rather than jittering.

I made the radius, offset, and rate adjustable so I could experiment. Smaller changes gave calm drifting motion, while larger ones made the boid turn more aggressively. I also added toroidal wrapping so movement could continue across screen edges without breaking flow.

```
// ---------- Wander ----------
float wanderRadius = 60.0f;
float wanderOffset = 40.0f;
float wanderRate = 6.0f;
```



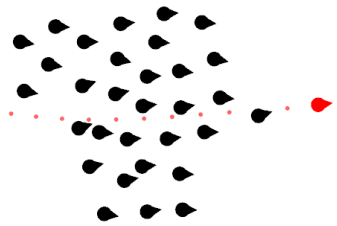
The last behavior that I implemented was Flocking. This one started similarly to Wander, since I reused that logic to drive the leader and simply drew it in a different color (red) so it was obvious who the group should follow. That part was straightforward. The difficult part was getting the followers to maintain good separation, stay cohesive, wrap correctly with the leader, and avoid getting ahead of it.

The followers blend several influences: attraction toward the leader, separation from nearby boids, and cohesion toward the local center of mass. I kept these as tunable weights, which turned out to be extremely important. Small changes could make the flock either collapse into a tight clump or spread too far apart. I also ran into issues where followers would turn the wrong direction when the leader crossed the screen boundary, so I had to account for toroidal distance and make sure they always pursued the shortest path.

To improve the overall look, I adjusted the followers to aim slightly behind the leader rather than directly at it. This helped keep the leader visually in front and made the movement feel more intentional. During polishing, I added the ability to spawn or remove boids at runtime, which helped me test how stable the system was at different densities. At higher numbers, especially up in the 1000s, my system starts to struggle with keeping the leader clearly ahead, which shows how sensitive flocking is to scaling and influence balance.

```
// ---------- Blend ----------
vec2 blend = wLeader * leaderDir + wSeparation * sep + wCohesion * coh;

if (length(blend) > 0.001f) {
    vec2 desiredVel = normalize(blend) * b.maxSpeed;
    vec2 steer = desiredVel - b.velocity;
    b.velocity += steer * dt * 1.8f;
}
```



One of the biggest things I learned from this assignment is that having tunable values allows you to play around and fine-tune the behaviors. Changing weights, radii, or timing by small amounts could completely change how the boids behaved. Once I started treating the system like something to tune instead of something to “finish,” the results improved a lot.

I also got better at understanding why the boids were doing what they were doing. Instead of randomly changing numbers, I could usually predict how increasing separation or leader influence would affect the motion. That shift from guessing to having an idea of the outcome felt like real progress. Reading the book and studying the pseudo-code and slides helped me understand the math and how velocity, position, and orientation all worked together to achieve the movement behaviors.